

// PRÁCTICA FINAL · 15 PUNTOS · 1ER REPORTE

HotelZormat: Sistema de Gestión

Aplicación de escritorio CRUD con SQL Server, WinForms y Repository Pattern

Asignatura:	ISW-123 · Programación Media	Modalidad:	Individual
Profesor:	Ing. Ivan Zorrilla	Valor:	15 puntos (1er Reporte)
Período:	Mayo-Junio 2026	Entrega:	Ver fecha en plataforma

Objetivo

Construir una aplicación de escritorio funcional para la gestión operativa de un hotel — **HotelZormat** — que demuestre dominio de los conceptos vistos durante el semestre: arquitectura por capas, manipulación de datos con SQL Server, estructuras de control (switch, foreach), captura de excepciones, y el patrón Repository sobre WinForms.

Contexto del proyecto

Zormat Technology Solutions ha sido contratada por un hotel boutique de la zona de Bávaro para construir un MVP que les permita administrar habitaciones y huéspedes, registrar estadías, y respetar las reglas tributarias dominicanas (ITBIS 18%, propina legal 10%, cédulas de 11 dígitos). Esta práctica cubre el **módulo base operativo** del sistema.

Lo que debes construir

Componente	Operaciones obligatorias
ACCESO Y SEGURIDAD	
Login y roles	Formulario de inicio de sesión contra tabla Usuarios. Al menos 2 roles activos (Administrador y Recepcionista) con permisos diferenciados — ejemplo: solo el Admin puede eliminar habitaciones. Contraseña almacenada con hash, no texto plano.
GESTIÓN OPERATIVA	
Dashboard de habitaciones	Vista principal tipo tablero con todas las habitaciones del hotel mostradas con colores por estado (verde=Disponible, rojo=Ocupada, naranja=Reservada, azul=Limpieza). Refresco automático al cambiar estado. Implementado con switch sobre el estado.
CRUD Habitaciones	Las 4 operaciones funcionando contra SQL: Listar con filtros por piso y estado, Crear , Actualizar (tipo, estado, capacidad), Eliminar (con confirmación previa). Llenado de ComboBox con foreach.

CRUD Huéspedes	Las 4 operaciones funcionando. Validación de cédula = 11 dígitos para dominicanos. Soporte para tipo de documento (Cédula / Pasaporte) ya que el hotel recibe turistas. Búsqueda por cédula o nombre. Historial de estadías del huésped.
CRUD Reservas	Crear reserva con fechas de check-in/check-out (validar que check-out > check-in). Estados: Pendiente, Confirmada, Cancelada. Calcular automáticamente el total de noches y monto. Listar reservas próximas (próximos 7 días).
FACTURACIÓN Y FLUJO HOTELERO	
Check-in / Check-out	Convertir una reserva confirmada en una estadía activa (check-in): cambia el estado de la habitación a Ocupada. El check-out cierra la estadía, genera la factura, y libera la habitación pasándola a estado Limpieza.
Facturación con NCF	Generar factura al cerrar la estadía: subtotal (noches × tarifa) + ITBIS 18% + propina legal 10% . Numeración secuencial de NCF (tipo Consumo Final). Mostrar la factura en pantalla con todos los desgloses. Guardar en BD.
Tarifas con temporada	Cada tipo de habitación tiene tarifa base. Aplicar factor por temporada con switch: Alta (sin descuento), Media (10% descuento), Baja (20% descuento). La temporada se selecciona al crear la reserva.
REPORTES Y CONTROL	
Reportes mínimos	Al menos 2 reportes visibles desde la UI: (1) Ocupación del día (lista de habitaciones ocupadas con huésped); (2) Ingresos por rango de fecha (suma de facturas entre dos fechas que el usuario seleccione).
Bitácora de acciones	Tabla Bitacora que registre quién hizo cada acción crítica (login, check-in, check-out, facturación) con fecha/hora. Vista para consultarla — solo accesible al rol Administrador.

Requisitos técnicos obligatorios

Stack:	C# 7.3 / .NET Framework 4.7.2 / Windows Forms / SQL Server (Express o superior). Prohibido usar features de C# 8 o superior.
Arquitectura:	El proyecto debe organizarse en al menos 3 capas separadas (HotelZormat.UI, HotelZormat.Negocio, HotelZormat.Datos) más un proyecto/carpeta Modelo para las clases de dominio.
Conexión a BD:	El connection string debe estar en App.config. No se acepta hardcodearlo en el código.
Repository Pattern:	Cada entidad principal debe tener su clase repositorio (HabitacionRepository, HuespedRepository, UsuarioRepository) que implemente operaciones CRUD con SqlConnection, SqlCommand y SqlDataReader.
Parámetros SQL:	Todas las queries deben usar parámetros @nombre con AddWithValue. Cero concatenación de strings en SQL.
Manejo de errores:	Cada event handler que llame a Negocio o Datos debe tener al menos un try/catch con catch específicos para FormatException, SQLException y Exception genérico (en ese orden). Mensajes amigables al usuario con MessageBox.
Excepción personalizada:	Al menos 1 excepción personalizada del negocio (ejemplo: HabitacionOcupadaException) con propiedad propia, atrapada específicamente en la UI.
Convenciones:	Controles WinForms nombrados con prefijo de tipo (txtNombre, btnGuardar, lblTotal, cboTipo, lstHabitaciones).
Base de datos:	Entregar el script SQL completo (CREATE DATABASE, CREATE TABLE de todas las tablas, INSERT de datos iniciales) como archivo .sql dentro del repositorio.

Rúbrica de evaluación · 15 puntos

Cada criterio se evalúa de forma independiente. El puntaje total se obtiene sumando lo alcanzado en cada uno. **No se otorga puntos por intención:** si el código no compila, no se evalúa esa funcionalidad.

#	Criterio	Qué se evalúa	Pts
1	Arquitectura por capas	Proyecto separado en UI / Negocio / Datos / Modelo. La UI no llama directamente a SqlConnection. HabitaciónService delega al Repository.	2.0
2	Base de datos	Script .sql con CREATE DATABASE, tablas correctamente tipadas (VARCHAR/NVARCHAR, INT, DECIMAL, DATETIME), PK definidas. INSERT de datos iniciales.	1.5
3	Conexión y App.config	Connection string en App.config. Helper centralizado. Sin hardcodear nada de credenciales en el código fuente.	0.5
4	CRUD Habitaciones	Las 4 operaciones funcionando contra la BD: Listar (con foreach para llenar UI), Crear, Actualizar, Eliminar. Vista visual con switch para colores por estado.	3.0
5	CRUD Huéspedes	Las 4 operaciones funcionando. Validación de cédula 11 dígitos. Soporte para cédula vs pasaporte. Búsqueda por cédula.	2.5
6	Login y roles	Formulario de login funcional contra tabla Usuarios. Diferenciación entre Administrador y Recepcionista (al menos un permiso distinto).	1.0
7	Registro de estadía	Asignar huésped a habitación disponible. Cálculo correcto: subtotal + ITBIS 18% + propina 10%. Cambio automático del estado de la habitación.	1.5
8	Manejo de excepciones	Cada operación crítica con try/catch específicos (FormatException, SQLException, Exception). Mensajes amigables. Mínimo 1 excepción personalizada del negocio atrapada en UI.	1.5
9	Seguridad SQL	Todas las queries con parámetros (@nombre + AddWithValue). Cero concatenación. Validaciones de entrada antes de hacer queries.	1.0
10	Calidad y entrega	Repositorio GitHub público con mínimo 10 commits descriptivos. README con instrucciones. Convención de nombres en controles. Código formateado.	0.5
	TOTAL		15.0

Forma de entrega

- Repositorio **GitHub público** con el código fuente completo del proyecto Visual Studio.
- El nombre del repositorio debe ser HotelZormat - [matrícula] (reemplazar con su matrícula real).
- Mínimo **10 commits descriptivos** a lo largo del desarrollo. No se acepta un solo commit final.
- Archivo README.md con: nombre completo, matrícula, instrucciones de configuración (cómo restaurar la BD, dónde está el App.config, qué cadena de conexión usar).
- Archivo script_bd.sql con CREATE DATABASE + CREATE TABLE + INSERT.
- Entregar el **link del repositorio** en la plataforma del aula virtual antes de la fecha límite.

Honestidad académica · Reglas anti-IA

OBLIGATORIO PARA CADA ESTUDIANTE

En al menos **una variable** de su código, deben usar su **matrícula real** como valor (ejemplo: `int matricula = 2023123456;`). En el comentario superior de **cada archivo .cs principal**, debe aparecer su número de cédula. El profesor revisará estos marcadores en cada proyecto.

El uso de IA generativa (ChatGPT, Claude, GitHub Copilot, etc.) para escribir código completo está **prohibido**. Sí se permite consultar documentación, foros, y videos. Si se detecta código generado por IA sin comprensión (no puede explicarlo en defensa oral), la calificación es **0 puntos**.

Cada estudiante debe estar preparado para una **defensa oral** de 5-10 minutos donde se le pedirá explicar partes específicas de su código (un catch, un foreach, una consulta SQL). Quien no pueda explicar lo que entregó, pierde el puntaje del componente correspondiente.

Fecha límite y penalizaciones

Entrega a tiempo	Calificación completa según rúbrica.
1-3 días tarde	Penalización de 3 puntos sobre el total obtenido.
Más de 3 días	No se acepta. Calificación de 0 puntos.
Sin GitHub público	No se evalúa. Calificación de 0 puntos.
Plagio detectado	0 puntos para ambos estudiantes involucrados + reporte a la Escuela.

Recursos disponibles

- Material visto en clase: Días 01 al 09 (HTMLs interactivos en la plataforma).
- Solución completa del formulario frmGestionHabitaciones del Día 08 (puede usarse como base de partida).
- Lab anterior "Peluquería El Cojo" donde ya vieron ADO.NET con Repository Pattern.

- Documentación oficial de Microsoft (link en plataforma del curso).
- Sesiones de consulta del profesor: durante las horas regulares de clase.

Lo importante no es la perfección, es la entrega.

Mejor entregar un CRUD que funcione completo en 2 entidades, que medio empezar 4. Si tienes dudas durante el desarrollo, escribe al profesor antes — no esperes al día de la entrega.